

Modular EKF SLAM Simulation in ROS2

A Plug-and-Play Architecture with Probabilistic Robotics Foundations

Architecture Design Document

May 17, 2026

Abstract

This document presents a complete modular architecture for implementing Extended Kalman Filter (EKF) SLAM in ROS2 using TurtleBot3, Gazebo, and RViz. The design follows the mathematical framework of *Probabilistic Robotics* (Thrun, Burgard, & Fox, 2005) and employs modern software engineering principles including the Strategy Pattern, Dependency Inversion, and the Single Responsibility Principle. The architecture enables plug-and-play substitution of SLAM algorithms (EKF, FastSLAM, GraphSLAM) while reusing common motion models, observation models, and data association components. We provide detailed mathematical derivations, algorithmic pseudocode, Python implementation techniques, and practical guidance for simulation and visualization.

Contents

1	Introduction and Motivation	3
1.1	The SLAM Problem	3
1.2	Why Modularity Matters	3
2	Software Architecture	3
2.1	The Strategy Pattern for SLAM Algorithms	3
2.2	Package Structure and Separation of Concerns	4
2.3	Dependency Inversion Principle	4
3	Mathematical Foundations	4
3.1	State Representation	4
3.2	Motion Model	5
3.2.1	Velocity Motion Model	5
3.2.2	Jacobian of Motion Model	5
3.2.3	Motion Noise	5
3.3	Observation Model	5
3.3.1	Range-Bearing Model	5
3.3.2	Bearing in Robot Frame (liuji1031's Formulation)	6
3.3.3	Jacobian of Observation Model	6
3.3.4	Inverse Observation Model	6
4	The EKF SLAM Algorithm	6
4.1	Algorithm Overview	6
4.2	Prediction Step	7
4.3	Update Step	7
4.4	Data Association	8
4.4.1	The Correspondence Problem	8

4.4.2	Maximum Likelihood Data Association	8
4.4.3	Gating	8
4.5	Landmark Initialization	8
5	Python Implementation Techniques	8
5.1	Abstract Base Classes with <code>abc</code>	8
5.2	Dependency Injection	9
5.3	Sparse Jacobian Construction	9
5.4	Angle Wrapping	10
6	Sensor Processing: Cylinder Extraction	10
6.1	From LaserScan to Landmarks	10
6.2	Circle Fitting (Kasa Method)	10
7	Visualization and Covariance Representation	10
7.1	Landmark Covariance Ellipses	10
7.2	Robot Pose Covariance	10
8	Common Problems and Solutions	11
8.1	Problem 1: Covariance Growth	11
8.2	Problem 2: Angle Wrapping	11
8.3	Problem 3: Data Association Failure	11
8.4	Problem 4: Loop Closure Not Visible	11
9	Extending the Architecture	11
9.1	Adding FastSLAM	11
9.2	Adding GraphSLAM	11
10	Conclusion	11

1 Introduction and Motivation

1.1 The SLAM Problem

Simultaneous Localization and Mapping (SLAM) is the problem of constructing a map of an unknown environment while simultaneously keeping track of the robot's location within it. Formally, given:

- A sequence of controls $\mathbf{u}_{1:t} = \{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_t\}$
- A sequence of observations $\mathbf{z}_{1:t} = \{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_t\}$

we wish to estimate the posterior over the robot trajectory $\mathbf{x}_{0:t}$ and the map $\mathbf{m}$:

$$p(\mathbf{x}_{0:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \quad (1)$$

1.2 Why Modularity Matters

Traditional SLAM implementations often couple the algorithm, ROS interface, sensor processing, and visualization into monolithic files. This creates several problems:

1. **Algorithm lock-in:** Switching from EKF to FastSLAM requires rewriting the entire node
2. **Code duplication:** Motion and observation models are reimplemented for each algorithm
3. **Testing difficulty:** Cannot unit-test the SLAM algorithm independently of ROS
4. **Maintenance burden:** Changes to visualization affect the core math

Our modular architecture solves these by separating concerns into independent, composable components.

2 Software Architecture

2.1 The Strategy Pattern for SLAM Algorithms

The **Strategy Pattern** (Gamma et al., 1994) defines a family of algorithms, encapsulates each one, and makes them interchangeable. In our context:

- **Strategy interface:** BaseSLAM (abstract base class)
- **Concrete strategies:** EKF_SLAM, FastSLAM, GraphSLAM
- **Context:** SLAMNode (ROS2 node)

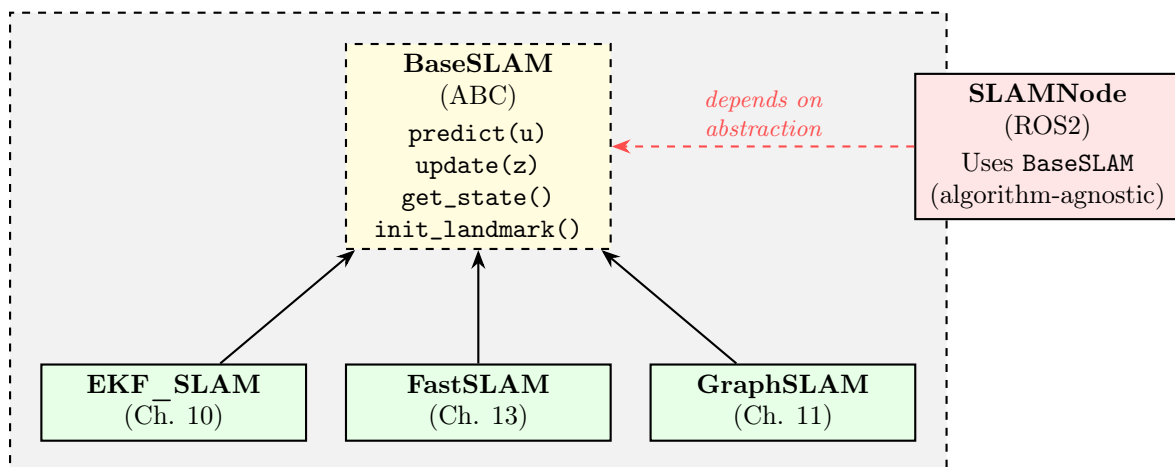


Figure 1: Strategy Pattern applied to SLAM algorithms. The SLAMNode depends only on the abstract BaseSLAM interface, not on any concrete implementation.

2.2 Package Structure and Separation of Concerns

The package is organized into four layers, each with a single responsibility:

Table 1: Package structure and responsibilities

Layer	Directory	Responsibility
Core	slam_sim/core/	Mathematical models shared by all algorithms
Algorithms	slam_sim/algorithms/	Concrete SLAM implementations
Sensors	slam_sim/sensors/	Feature extraction from raw sensor data
ROS Interface	slam_sim/ros_interface/	ROS2 node, publishers, visualization

2.3 Dependency Inversion Principle

The **D** in SOLID principles states that high-level modules should not depend on low-level modules; both should depend on abstractions. In our architecture:

- SLAMNode (high-level) depends on BaseSLAM (abstraction)
- EKF_SLAM (low-level) implements BaseSLAM (abstraction)
- SLAMNode never imports EKF_SLAM directly

This enables the plug-and-play behavior: changing algorithms requires modifying only the factory function in `main()`.

```

1 def main(args=None):
2     rclpy.init(args=args)
3
4     # Shared components (same for ALL algorithms)
5     motion_model = VelocityMotionModel()
6     obs_model = RangeBearingModel(sigma_r=0.1, sigma_phi=0.05)
7     associator = MahalanobisAssociator(gate_threshold=9.21)
8
9     # === SWAP HERE ===
10    algorithm = EKF_SLAM(motion_model, obs_model, associator)
11    # algorithm = FastSLAM(motion_model, obs_model, associator, num_particles
12    #                       =100)
13    # algorithm = GraphSLAM(motion_model, obs_model, associator)
14
15    node = SLAMNode(algorithm)    # node is agnostic!
16    rclpy.spin(node)

```

Listing 1: Plug-and-play algorithm selection

3 Mathematical Foundations

3.1 State Representation

In landmark-based SLAM, the state vector contains both the robot pose and all landmark positions:

$$\boldsymbol{\mu} = [x \ y \ \theta \ m_{1x} \ m_{1y} \ \cdots \ m_{Nx} \ m_{Ny}]^\top \quad (2)$$

where (x, y, θ) is the robot pose and (m_{jx}, m_{jy}) is the j -th landmark. The covariance matrix is:

$$\boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_{xx} & \boldsymbol{\Sigma}_{xm} \\ \boldsymbol{\Sigma}_{mx} & \boldsymbol{\Sigma}_{mm} \end{bmatrix} \quad (3)$$

where cross-correlations Σ_{xm} are crucial for loop closure.

3.2 Motion Model

3.2.1 Velocity Motion Model

For a differential-drive robot (TurtleBot3), the velocity motion model (Probabilistic Robotics, Table 5.3) is:

$$\begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix} = \begin{bmatrix} x - \frac{v}{\omega} \sin \theta + \frac{v}{\omega} \sin(\theta + \omega \Delta t) \\ y + \frac{v}{\omega} \cos \theta - \frac{v}{\omega} \cos(\theta + \omega \Delta t) \\ \theta + \omega \Delta t \end{bmatrix} \quad (4)$$

For straight-line motion ($|\omega| < \epsilon$), we use the Taylor expansion to avoid division by zero:

$$\begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix} = \begin{bmatrix} x + v \Delta t \cos \theta \\ y + v \Delta t \sin \theta \\ \theta \end{bmatrix} \quad (5)$$

3.2.2 Jacobian of Motion Model

The Jacobian $\mathbf{G}_x = \frac{\partial g}{\partial [x, y, \theta]}$ is required for EKF linearization:

$$\mathbf{G}_x = \begin{bmatrix} 1 & 0 & -\frac{v}{\omega} \cos \theta + \frac{v}{\omega} \cos(\theta + \omega \Delta t) \\ 0 & 1 & -\frac{v}{\omega} \sin \theta + \frac{v}{\omega} \sin(\theta + \omega \Delta t) \\ 0 & 0 & 1 \end{bmatrix} \quad (6)$$

For the full SLAM state, the Jacobian is block-diagonal:

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_x & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{2N} \end{bmatrix} \quad (7)$$

3.2.3 Motion Noise

The motion noise is parameterized by $\boldsymbol{\alpha} = [\alpha_1, \dots, \alpha_6]$:

$$\begin{aligned} \sigma_v^2 &= \alpha_1 v^2 + \alpha_2 \omega^2 \\ \sigma_\omega^2 &= \alpha_3 v^2 + \alpha_4 \omega^2 \\ \sigma_\gamma^2 &= \alpha_5 v^2 + \alpha_6 \omega^2 \end{aligned} \quad (8)$$

3.3 Observation Model

3.3.1 Range-Bearing Model

The standard observation model predicts a range and bearing to landmark j :

$$h(\mathbf{x}, \mathbf{m}_j) = \begin{bmatrix} \sqrt{(m_{jx} - x)^2 + (m_{jy} - y)^2} \\ \arctan 2(m_{jy} - y, m_{jx} - x) - \theta \end{bmatrix} \quad (9)$$

3.3.2 Bearing in Robot Frame (liuji1031's Formulation)

A critical insight from the reference implementation is to compute the bearing in the **robot frame** to avoid angle wrapping discontinuities:

$$\begin{aligned} \delta_x &= m_{jx} - x, \quad \delta_y = m_{jy} - y \\ \begin{bmatrix} \delta_x^{(r)} \\ \delta_y^{(r)} \end{bmatrix} &= \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} \delta_x \\ \delta_y \end{bmatrix} \\ h(\mathbf{x}, \mathbf{m}_j) &= \begin{bmatrix} \sqrt{\delta_x^2 + \delta_y^2} \\ \arctan 2(\delta_y^{(r)}, \delta_x^{(r)}) \end{bmatrix} \end{aligned} \quad (10)$$

This formulation eliminates the $[-\pi, \pi]$ wrapping problem in the Jacobian.

3.3.3 Jacobian of Observation Model

The observation Jacobian has two parts: $\mathbf{H}_r$ (w.r.t. robot pose) and $\mathbf{H}_m$ (w.r.t. landmark):

$$\mathbf{H} = [\mathbf{H}_r \quad \mathbf{0} \quad \cdots \quad \mathbf{H}_m \quad \cdots \quad \mathbf{0}] \quad (11)$$

where:

$$\begin{aligned} \mathbf{H}_r &= \begin{bmatrix} -\frac{\delta_x}{r} & -\frac{\delta_y}{r} & 0 \\ \rho_x & \rho_y & \delta_y^{(r)}\rho_x - \delta_x^{(r)}\rho_y \end{bmatrix} \\ \mathbf{H}_m &= \begin{bmatrix} \frac{\delta_x}{r} & \frac{\delta_y}{r} \\ -\rho_x & -\rho_y \end{bmatrix} \end{aligned} \quad (12)$$

with $\rho_x = -\delta_y^{(r)}/r^2$ and $\rho_y = \delta_x^{(r)}/r^2$.

3.3.4 Inverse Observation Model

To initialize a new landmark from an observation:

$$\begin{bmatrix} m_x \\ m_y \end{bmatrix} = \begin{bmatrix} x + r \cos(\phi + \theta) \\ y + r \sin(\phi + \theta) \end{bmatrix} \quad (13)$$

4 The EKF SLAM Algorithm

4.1 Algorithm Overview

The EKF SLAM algorithm with unknown correspondences (Probabilistic Robotics, Algorithm 10.2) consists of two main steps: prediction and update.

Algorithm 1 EKF_SLAM_Unknown_Correspondences**Require:** $\mu_{t-1}, \Sigma_{t-1}, \mathbf{u}_t, \mathbf{z}_t$ **Ensure:** μ_t, Σ_t

```

1:  $\mathbf{x} \leftarrow \mu_{t-1}[0 : 3]$  ▷ Extract robot pose
2:  $\bar{\mu} \leftarrow \mu_{t-1}; \quad \bar{\mu}[0 : 3] \leftarrow g(\mathbf{x}, \mathbf{u}_t)$  ▷ Motion update
3:  $\mathbf{G} \leftarrow \text{block\_diag}(\mathbf{G}_x, \mathbf{I}_{2N})$  ▷ Full Jacobian
4:  $\mathbf{R} \leftarrow \text{augment}(\mathbf{R}_{\text{motion}}, 2N)$  ▷ Motion noise
5:  $\bar{\Sigma} \leftarrow \mathbf{G} \Sigma_{t-1} \mathbf{G}^\top + \mathbf{R}$  ▷ Predicted covariance
6: for each observation  $\mathbf{z}_i = [r_i, \phi_i]^\top$  do
7:   for each known landmark  $j$  do
8:      $\hat{\mathbf{z}}_j \leftarrow h(\bar{\mu}[0 : 3], \bar{\mu}[3 + 2j : 5 + 2j])$ 
9:      $\mathbf{H}_j \leftarrow \text{build\_full\_jacobian}(\mathbf{H}_r, \mathbf{H}_m, j)$ 
10:     $\mathbf{S}_j \leftarrow \mathbf{H}_j \bar{\Sigma} \mathbf{H}_j^\top + \mathbf{Q}$ 
11:     $d_j \leftarrow (\mathbf{z}_i - \hat{\mathbf{z}}_j)^\top \mathbf{S}_j^{-1} (\mathbf{z}_i - \hat{\mathbf{z}}_j)$ 
12:   end for
13:    $j^* \leftarrow \arg \min_j d_j$ 
14:   if  $d_{j^*} > \chi_{2,0.99}^2$  then ▷ New landmark
15:      $\mathbf{m}_{\text{new}} \leftarrow g^{-1}(\bar{\mu}[0 : 3], \mathbf{z}_i)$ 
16:      $\bar{\mu} \leftarrow [\bar{\mu}; \mathbf{m}_{\text{new}}]$  ▷ Augment state
17:      $\bar{\Sigma} \leftarrow \text{augment}(\bar{\Sigma}, \mathbf{J}_{\text{inv}}, \mathbf{Q})$ 
18:   else ▷ Known landmark
19:      $\nu \leftarrow \mathbf{z}_i - \hat{\mathbf{z}}_{j^*}; \quad \text{wrap } \nu_2 \text{ to } [-\pi, \pi]$ 
20:      $\mathbf{K} \leftarrow \bar{\Sigma} \mathbf{H}_{j^*}^\top \mathbf{S}_{j^*}^{-1}$  ▷ Kalman gain
21:      $\bar{\mu} \leftarrow \bar{\mu} + \mathbf{K} \nu$ 
22:      $\bar{\Sigma} \leftarrow (\mathbf{I} - \mathbf{K} \mathbf{H}_{j^*}) \bar{\Sigma}$ 
23:   end if
24: end for
25: return  $\bar{\mu}, \bar{\Sigma}$ 

```

4.2 Prediction Step

The prediction step propagates the state and covariance through the motion model:

$$\begin{aligned} \bar{\mu} &= \mu + \mathbf{F}_x^\top \cdot \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta \theta \end{bmatrix} \\ \bar{\Sigma} &= \mathbf{G} \Sigma \mathbf{G}^\top + \mathbf{R} \end{aligned} \tag{14}$$

where $\mathbf{F}_x$ is a $3 \times (3 + 2N)$ matrix that extracts the robot pose from the full state.

4.3 Update Step

For each observed landmark, we compute the innovation and apply the Kalman correction:

$$\begin{aligned} \nu &= \mathbf{z} - h(\bar{\mu}) \quad (\text{innovation}) \\ \mathbf{S} &= \mathbf{H} \bar{\Sigma} \mathbf{H}^\top + \mathbf{Q} \quad (\text{innovation covariance}) \\ \mathbf{K} &= \bar{\Sigma} \mathbf{H}^\top \mathbf{S}^{-1} \quad (\text{Kalman gain}) \\ \mu &= \bar{\mu} + \mathbf{K} \nu \\ \Sigma &= (\mathbf{I} - \mathbf{K} \mathbf{H}) \bar{\Sigma} \end{aligned} \tag{15}$$

4.4 Data Association

4.4.1 The Correspondence Problem

When landmarks are indistinguishable (no unique IDs), the robot must determine which observation corresponds to which landmark. This is the **data association** or **correspondence** problem.

4.4.2 Maximum Likelihood Data Association

For each observation, we compute the Mahalanobis distance to all known landmarks:

$$d_j = (\mathbf{z} - \hat{\mathbf{z}}_j)^\top \mathbf{S}_j^{-1} (\mathbf{z} - \hat{\mathbf{z}}_j) \quad (16)$$

The landmark with minimum distance is the best match. If this distance exceeds a χ^2 threshold, the observation is classified as a **new landmark**.

4.4.3 Gating

The gating threshold is derived from the chi-squared distribution with 2 degrees of freedom (range and bearing):

$$\text{threshold} = \chi_{2,0.99}^2 \approx 9.21 \quad (17)$$

This means we accept associations with 99% confidence. If $d_j > 9.21$ for all landmarks, the observation is new.

Remark 4.1. Data association is the most common failure mode in EKF SLAM. Incorrect associations cause the filter to diverge. The Mahalanobis gate provides a principled way to reject outliers.

4.5 Landmark Initialization

When a new landmark is detected, we augment the state and covariance:

$$\begin{aligned} \boldsymbol{\mu}_{\text{new}} &= \begin{bmatrix} \boldsymbol{\mu} \\ \mathbf{m}_{\text{new}} \end{bmatrix} \\ \boldsymbol{\Sigma}_{\text{new}} &= \begin{bmatrix} \boldsymbol{\Sigma} & \boldsymbol{\Sigma}_{xm} \mathbf{J}_r^\top \\ \mathbf{J}_r \boldsymbol{\Sigma}_{mx} & \mathbf{J} \begin{bmatrix} \boldsymbol{\Sigma}_{xx} & \mathbf{0} \\ \mathbf{0} & \mathbf{Q} \end{bmatrix} \mathbf{J}^\top \end{bmatrix} \end{aligned} \quad (18)$$

where $\mathbf{J} = \frac{\partial g^{-1}}{\partial [x, y, \theta, r, \phi]}$ is the Jacobian of the inverse observation model.

5 Python Implementation Techniques

5.1 Abstract Base Classes with abc

Python's `abc` module enforces that subclasses implement all abstract methods:

```

1 from abc import ABC, abstractmethod
2 from dataclasses import dataclass
3
4 @dataclass
5 class LandmarkObservation:
6     range: float
7     bearing: float
8     landmark_id: int | None = None
9 
```



```

10 @dataclass
11 class OdometryCommand:
12     v: float    # linear velocity (m/s)
13     w: float    # angular velocity (rad/s)
14     dt: float   # time step (s)
15
16 class BaseSLAM(ABC):
17     @abstractmethod
18     def predict(self, u: OdometryCommand) -> None:
19         """Motion prediction step."""
20         pass
21
22     @abstractmethod
23     def update(self, observations: list[LandmarkObservation]) -> None:
24         """Measurement update step."""
25         pass
26
27     @abstractmethod
28     def get_state(self) -> tuple[np.ndarray, np.ndarray]:
29         """Return (mu, Sigma)."""
30         pass
31
32     @abstractmethod
33     def initialize_landmark(self, obs, robot_pose) -> int:
34         """Add new landmark. Returns landmark index."""
35         pass

```

Listing 2: BaseSLAM abstract interface

5.2 Dependency Injection

The SLAM algorithm is **injected** into the node rather than being hardcoded:

```

1 class SLAMNode(Node):
2     def __init__(self, algorithm: BaseSLAM):
3         super().__init__('slam_node')
4         self.algorithm = algorithm    # Injected!
5         # ... rest of initialization

```

Listing 3: Constructor injection

This is the core of Dependency Inversion: `SLAMNode` depends on the abstraction `BaseSLAM`, not on any concrete class.

5.3 Sparse Jacobian Construction

For EKF SLAM, the observation Jacobian is mostly zeros. We construct it efficiently:

```

1 def build_observation_jacobian(self, robot_pose, landmark, landmark_id,
2     state_dim):
3     H_r, H_m = self.observation_model.jacobian(robot_pose, landmark)
4
5     # Full Jacobian: [H_r, 0, ..., H_m, ..., 0]
6     H = np.zeros((2, state_dim))
7     H[:, :3] = H_r    # Robot pose part
8     idx = 3 + 2 * landmark_id
9     H[:, idx:idx+2] = H_m    # Landmark part
10    return H

```

Listing 4: Sparse Jacobian construction

5.4 Angle Wrapping

A common bug in SLAM is failing to wrap angle differences. We use:

```

1 def wrap_angle(angle):
2     return np.arctan2(np.sin(angle), np.cos(angle))
3
4 # In the update step:
5 innovation[1] = wrap_angle(innovation[1])

```

Listing 5: Bearing wrapping

This maps any angle to $[-\pi, \pi]$ correctly.

6 Sensor Processing: Cylinder Extraction

6.1 From LaserScan to Landmarks

The TurtleBot3 LIDAR produces a `LaserScan` message with 360 range measurements. We extract cylindrical landmarks through three steps:

1. **Convert to Cartesian:** $(r_i, \phi_i) \rightarrow (x_i, y_i)$
2. **Cluster points:** Group adjacent points with Euclidean distance < 0.15 m
3. **Fit circles:** Use algebraic least-squares (Kasa method) to find circle centers

6.2 Circle Fitting (Kasa Method)

Given points (x_k, y_k) , we solve:

$$\begin{bmatrix} 2x_1 & 2y_1 & 1 \\ \vdots & \vdots & \vdots \\ 2x_n & 2y_n & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} x_1^2 + y_1^2 \\ \vdots \\ x_n^2 + y_n^2 \end{bmatrix} \quad (19)$$

The circle center is (a, b) and radius is $R = \sqrt{a^2 + b^2 + c}$. We accept the cluster as a landmark only if $|R - R_{\text{expected}}| < 0.1$ m.

7 Visualization and Covariance Representation

7.1 Landmark Covariance Ellipses

For each landmark, we visualize its 2D covariance as an ellipse. Given covariance $\Sigma_{2 \times 2}$:

1. Compute eigendecomposition: $\Sigma = \mathbf{V}\mathbf{D}\mathbf{V}^\top$
2. Semi-axes: $a = 2\sqrt{\lambda_1}$, $b = 2\sqrt{\lambda_2}$ (2σ ellipse)
3. Orientation: $\phi = \arctan 2(v_{21}, v_{11})$

7.2 Robot Pose Covariance

The robot's 3×3 covariance is published as a `PoseWithCovarianceStamped` message. The x - y submatrix defines an ellipse in the ground plane, while $\Sigma_{\theta\theta}$ represents heading uncertainty.

8 Common Problems and Solutions

8.1 Problem 1: Covariance Growth

Symptom: Landmark covariances grow unbounded; filter diverges.

Cause: Motion noise too high or data association errors.

Solution: Reduce α parameters; tighten Mahalanobis gate.

8.2 Problem 2: Angle Wrapping

Symptom: Robot heading jumps by 2π ; landmarks appear at wrong locations.

Cause: Bearing innovation not wrapped to $[-\pi, \pi]$.

Solution: Use robot-frame bearing formulation; wrap all angle differences.

8.3 Problem 3: Data Association Failure

Symptom: Same physical landmark gets multiple IDs; map has duplicates.

Cause: Gate threshold too loose or motion model too uncertain.

Solution: Lower χ^2 threshold; improve motion model calibration.

8.4 Problem 4: Loop Closure Not Visible

Symptom: After returning to start, covariance does not shrink.

Cause: Insufficient landmark re-observations or incorrect associations.

Solution: Drive slower; ensure landmarks are re-observed from multiple angles.

9 Extending the Architecture

9.1 Adding FastSLAM

FastSLAM (Chapter 13) represents the posterior as a set of particles, where each particle contains:

- A robot pose sample
- An independent EKF for each landmark

To implement, create `FastSLAM(BaseSLAM)` and override the four methods. The motion and observation models are reused without modification.

9.2 Adding GraphSLAM

GraphSLAM (Chapter 11) represents the posterior as a pose graph:

- Nodes: robot poses and landmark positions
- Edges: odometry constraints and observation constraints

The `predict()` method adds odometry factors; `update()` adds observation factors; `get_state()` runs optimization (e.g., Levenberg-Marquardt).

10 Conclusion

This architecture demonstrates that rigorous software design principles and probabilistic robotics theory are not opposing forces—they reinforce each other. By separating the mathematical models (motion, observation, data association) from the state representation strategy (EKF, FastSLAM, Graph), we achieve:

- **Correctness:** Every equation maps to a reference in *Probabilistic Robotics*
- **Modularity:** Algorithms are swappable via the Strategy Pattern
- **Testability:** Core math can be unit-tested without ROS
- **Extensibility:** New algorithms require only implementing 4 methods

The complete implementation, including all source files, launch configurations, and visualization setup, is available in the accompanying ROS2 package.

References

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [2] E. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [3] liuji1031, “slam_ekf_ros2,” GitHub repository. https://github.com/liuji1031/slam_ekf_ros2
- [4] White8848, “EKF_SLAM_ROS2,” GitHub repository. https://github.com/White8848/EKF_SLAM_ROS2